SOFTWARE PROJECT INSPECTION FORM

PROJECT INFORMATION

Reviewed Group: Team 3: Bytes & Fabric Devs

Project Name: UniStock

Reviewing Group: Team 4: Runtime Dynamic

Inspection Date: 29/10/2026

GitHub Repository: https://github.com/AlgoFuerte/UniStock

EVALUATION CRITERIA

01. GitHub: Edison — 100 points

Requirement: Review the GitHub repository according to Edison's criterion.

Score: _____ / 100

Observations:

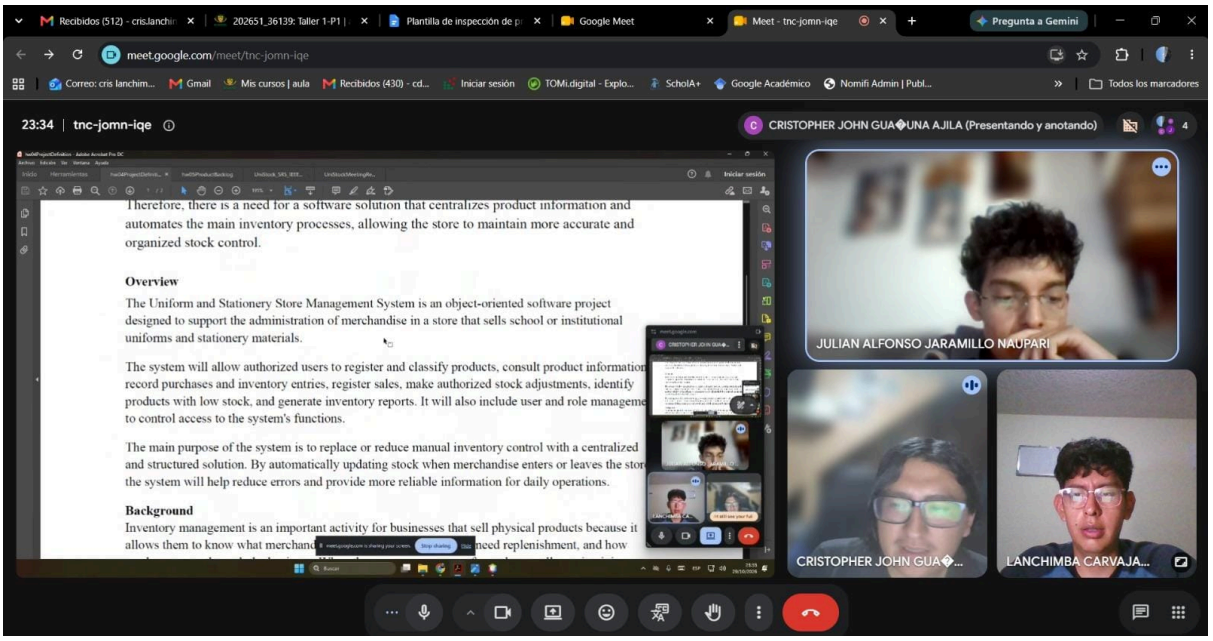
02. Definition — 10 points

Requirement: The project definition is clear and self-explanatory.

Score: **10** / 10

The document clearly explains the business, the inventory problem, and the proposed solution. It is easy to understand without extra explanations.

Evidence:



University of the Armed Forces "ESPE"

Project's Name: UniStock
Team's Name: Byte y Fabric Devs
Integrants: Criollo Dennis, Cuichan Leslye, Demera Wilkerfon

Problem

The uniform and stationery store currently does not have a computerized mechanism that allows it to properly manage and control its inventory. Product information and stock quantities may be handled manually, making it difficult to know the exact number of uniforms and stationery materials available at any given time.

This lack of inventory control can cause problems such as inaccurate stock information, difficulty identifying products that are running low, errors when recording merchandise entries and sales, and unnecessary time spent checking products manually. It can also make decision-making more difficult because the store does not have reliable and centralized information about its merchandise.

Therefore, there is a need for a software solution that centralizes product information and automates the main inventory processes, allowing the store to maintain more accurate and organized stock control.

Overview

The Uniform and Stationery Store Management System is an object-oriented software project designed to support the administration of merchandise in a store that sells school or institutional uniforms and stationery materials.

The system will allow authorized users to register and classify products, consult product information, record purchases and inventory entries, register sales, make authorized stock adjustments, identify products with low stock, and generate inventory reports. It will also include user and role management to control access to the system's functions.

The main purpose of the system is to replace or reduce manual inventory control with a centralized and structured solution. By automatically updating stock when merchandise enters or leaves the store, the system will help reduce errors and provide more reliable information for daily operations.

Background

Inventory management is an important activity for businesses that sell physical products because it allows them to know what merchandise is available, what products need replenishment, and how products move through the business. When these processes are performed manually, maintaining accurate information can become increasingly difficult as the number of products and transactions grows.

Observations:

- The definition clearly explains the business, the problem, and the purpose of UniStock. The reader can understand the proposed solution without extra information. We recommend separating the current prototype features from future features.

03. List of Possible Objects — 10 points

Requirement: At least 10 objects are clearly defined.

Score: 8 / 10

The document lists ten project elements and briefly explains their purpose. It also identifies their related test classes. We recommend separating business classes, enums, services, and utility classes. Adding key attributes and methods would make each description clearer.

Evidence:

10 objects and components from the project available for evaluation

- Product / ProductTest: Product model object and its corresponding unit test class to evaluate its behavior.
- ProductCategory / ProductCategoryTest: Product category object and its associated tests to verify classification.
- Role / RoleTest: Object that manages user roles and its permission validation tests.
- Sale / SaleTest: Object responsible for recording sales and its test suite to ensure total calculations.
- SaleDetail / SaleDetailTest: Object that stores details for each sale and its respective data evaluation.
- Stationery / StationeryTest: Object representing stationery items and their specific unit tests.
- Uniform / UniformTest: Object modeling school uniforms and its respective test file.
- User / UserTest: System user management object and its authentication or data tests.
- ProductService / ProductServiceTest: Business logic object (service) for product administration and its functional tests.
- FileHandler / FileHandlerTest: Structural object for file handling and persistence, evaluated through its read/write tests.

Observations:

- The diagram shows the main business classes and their relationships. It demonstrates good work in object-oriented modeling. We recommend correcting the inheritance symbols and matching the class names, attributes, and data types with the Java code.

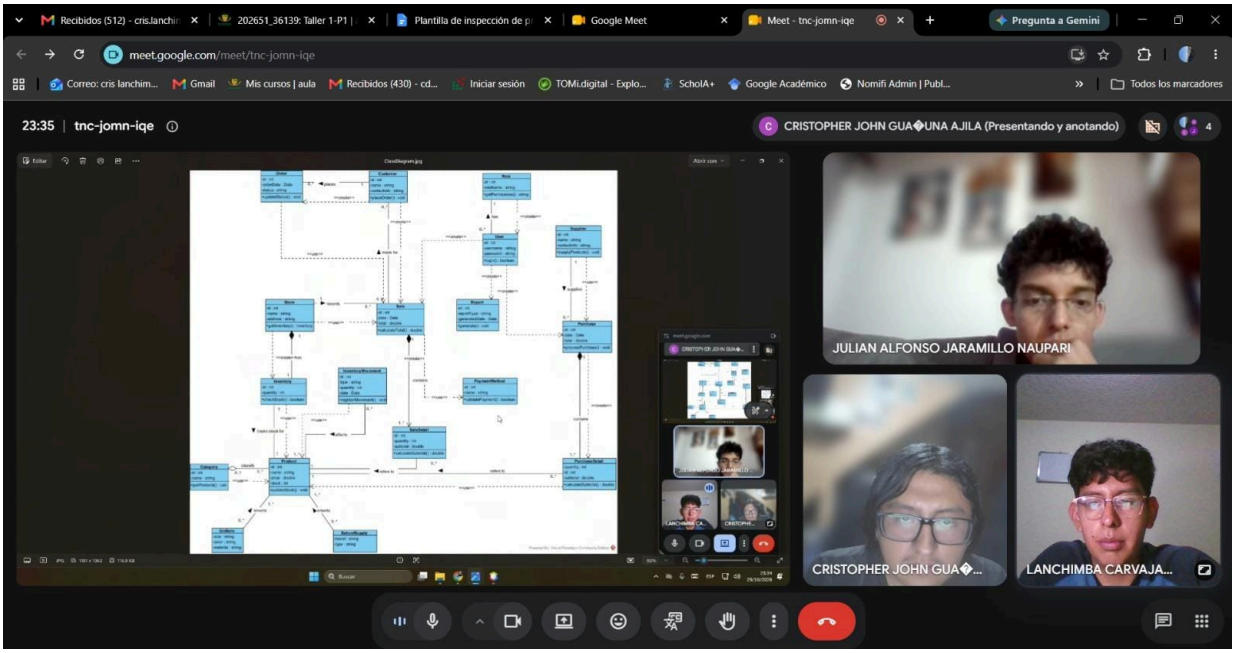
04. UML Class Diagram — 10 points

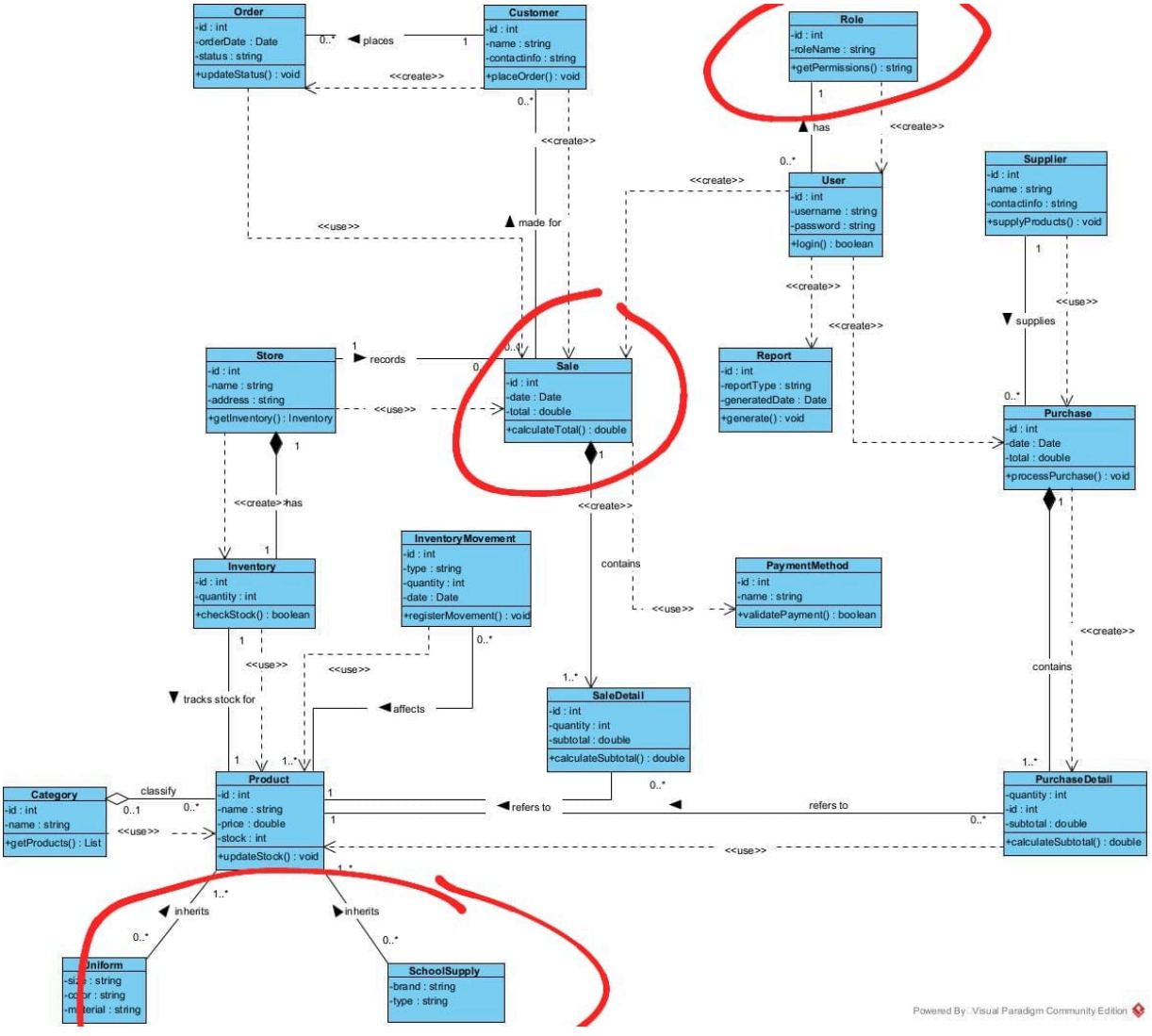
Requirement: The diagram shows the classes and their relationships.

Score: 8 / 10

The diagram includes classes, attributes, methods, relationships, and multiplicities. It helps explain the business structure.

Evidence:


meet.google.com/meet/tnc-jomn-iqe
23:35 | tnc-jomn-iqe
CRISTOPHER JOHN GUA♦UNA AJILA (Presentando y anotando)
JULIAN ALFONSO JARAMILLO NAUPARI
CRISTOPHER JOHN GUA♦...
LANCHIMBA CARVAJA...


Order
-id : int
-orderDate : Date
-status : string
+updateStatus() : void
Customer
-id : int
-name : string
-contactInfo : string
+placeOrder() : void
places
<<create>>
Role
-id : int
-roleName : string
+getPermissions() : string
has
User
-id : int
-username : string
-password : string
+login() : boolean
Supplier
-id : int
-name : string
-contactInfo : string
+supplyProducts() : void
supplies
made for
<<use>>
Store
-id : int
-name : string
-address : string
+getInventory() : Inventory
records
Sale
-id : int
-date : Date
-total : double
+calculateTotal() : double
Report
-id : int
-reportType : string
-generatedDate : Date
+generate() : void
Purchase
-id : int
-date : Date
-total : double
+processPurchase() : void
Inventory
-id : int
-quantity : int
+checkStock() : boolean
InventoryMovement
-id : int
-type : string
-quantity : int
-date : Date
+registerMovement() : void
PaymentMethod
-id : int
-name : string
+validatePayment() : boolean
contains
tracks stock for
affects
SaleDetail
-id : int
-quantity : int
-subtotal : double
+calculateSubtotal() : double
Category
-id : int
-name : string
+getProducts() : List
classify
Product
-id : int
-name : string
-price : double
-stock : int
+updateStock() : void
refers to
PurchaseDetail
-quantity : int
-id : int
-subtotal : double
+calculateSubtotal() : double
inherits
Uniform
SchoolSupply
-brand : string
-type : string
Powered By: Visual Paradigm Community Edition

Observations:

- The diagram shows the main business classes and their relationships. It demonstrates good work in object-oriented modeling. We recommend correcting the inheritance symbols and matching the class names, attributes, and data types with the Java code.

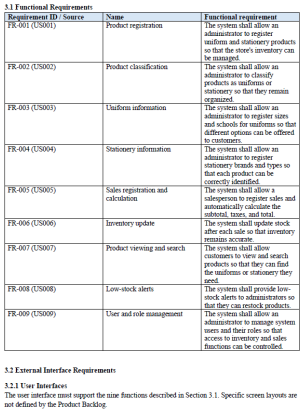
05. SRS — IEEE 830 — 10 points

Requirement: The SRS document is complete according to IEEE 830.

Score: **9** / 10

Complete and well-organized SRS. It follows IEEE 830, all user stories are covered, and the traceability is clear

Evidence



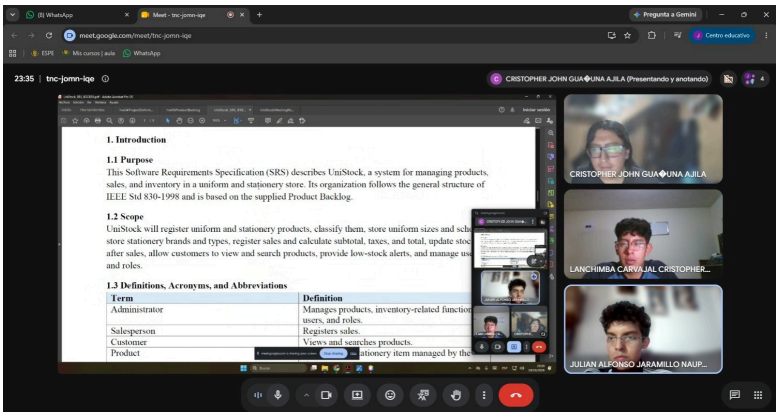
3.1 Functional Requirements

Requirement ID / Source	Name	Functional requirement
FR-001 (US001)	Product registration	The system shall allow an administrator to register uniform and stationery products so that the store's inventory can be managed.
FR-002 (US002)	Product classification	The system shall allow an administrator to classify products as uniforms or stationery so that they remain organized.
FR-003 (US003)	Uniform information	The system shall allow an administrator to register sizes and schools for uniforms so that different options can be offered to customers.
FR-004 (US004)	Stationery information	The system shall allow an administrator to register stationery brands and types so that each product can be correctly identified.
FR-005 (US005)	Sales registration and calculation	The system shall allow a salesperson to register sales and automatically calculate the subtotal, taxes, and total.
FR-006 (US006)	Inventory update	The system shall update stock after each sale so that inventory remains accurate.
FR-007 (US007)	Product viewing and search	The system shall allow customers to view and search products so that they can find the uniforms or stationery they need.
FR-008 (US008)	Low-stock alerts	The system shall provide low-stock alerts to administrators so that they can restock products.
FR-009 (US009)	User and role management	The system shall allow an administrator to manage system users and their roles so that access to inventory and sales functions can be controlled.

3.2 External Interface Requirements

3.2.1 User Interfaces

The user interface must support the nine functions described in Section 3.1. Specific screen layouts are not defined by the Product Backlog.



Observations: Could be improved by adding more specific numbers to the non-functional requirements and clearer rules for taxes and low-stock alerts.

06. Product Backlog — 10 points

Requirement: At least 9 features perform calculations.

Number of features perform calculations. **3**/9
But this parameter does not affect the total note

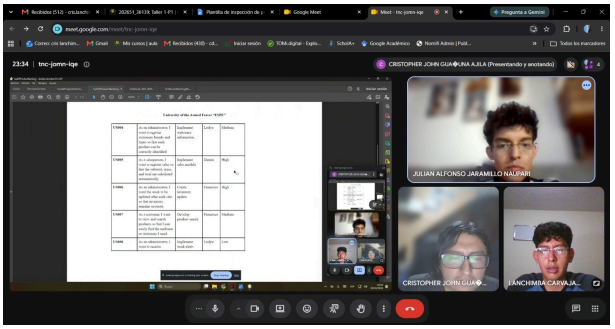
Score: **10**/ 10

The backlog is well organized and easy to read. It contains exactly 9 user stories (US001–US009), each with a clear ID, task name, task owner, and priority. The stories are understandable and cover different areas of the store.

Evidence:

University of the Armed Forces "ESPE"

US004	As an administrator, I want to register stationery brands and types so that each product can be correctly identified.	Implement stationery information	Leslye	Medium
US005	As a salesperson, I want to register sales so that the subtotal, taxes, and total are calculated automatically.	Implement sales module	Dennis	High
US006	As an administrator, I want the stock to be updated after each sale so that inventory remains accurate.	Create inventory update	Francisco	High
US007	As a customer, I want to view and search products so that I can easily find the uniforms or stationery I need.	Develop product search	Francisco	Medium
US008	As an administrator, I want to receive	Implement stock alerts	Leslye	Low



Observations: The requirement "at least 9 features that compute something" is not met. Only 2 or 3 of the 9 stories actually calculate anything

07. UML Use Case Diagrams — 10 points

Requirement: The diagrams show the use cases and actors.

Score: 9/ 10

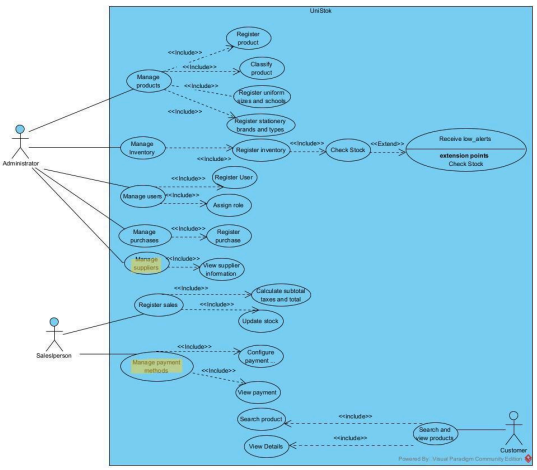
The diagram correctly covers most of the backlog: Administrator manages products, inventory, alerts, and users; Salesperson registers sales and updates stock; Customer searches and views products. Clear and well organized.

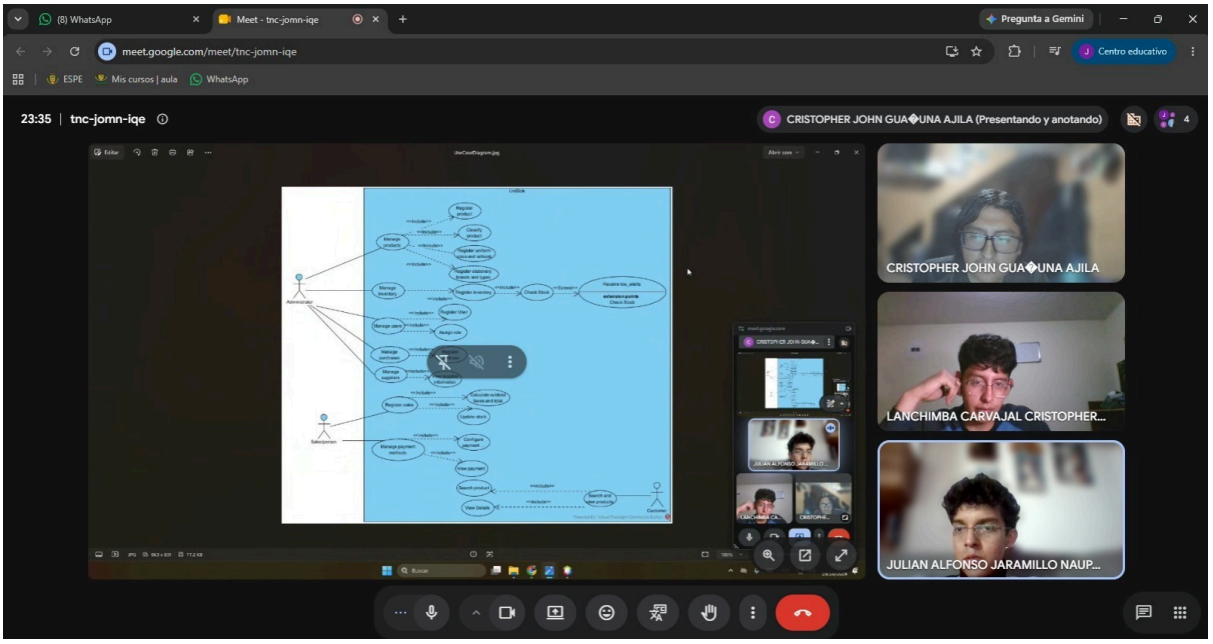
Evidence

University of the Armed Forces "ESPE"

From a software engineering perspective, Object-Oriented Programming provides an appropriate approach for modeling this type of system because the real-world elements involved in the business can be represented as objects. Products, users, sales, inventory movements and reports can be represented by classes with attributes and behaviors. Concepts such as encapsulation, inheritance, abstraction and polymorphism can therefore be applied in the development of the project.

The proposed system will serve as an academic OOP project while also representing a practical solution to the store's inventory-control problem. Its design can be expanded in the future with additional features such as suppliers, customers, payment methods, barcode scanning, purchase orders and more advanced sales reports.





Observations: The diagram includes the "Manage suppliers" and "Manage payment methods" modules, which are not in the product backlog or the SRS. The Project Definition only mentions suppliers and payment methods as future expansions

08. CSV Data Reports — 10 points

Requirement: The system generates at least 2 CSV data reports.

Score: **9** / 10

The system generates al least 2 CSV data reports one for users and another for sales

Evidence

```
--- GESTION DE USUARIOS ---
1. Registrar nuevo usuario
2. Listar usuarios
3. Exportar reporte de usuarios a CSV
Opcion: 3
Nombre del archivo CSV (ej: usuarios.csv): Usuarios reporte
Reporte CSV generado exitosamente en: Usuarios reporte.csv
Proceso de exportacion de usuarios finalizado.

--- CONSULTA DE VENTAS ---
1. Listar historial de ventas
2. Exportar reporte de ventas a CSV
Opcion: 2
Nombre del archivo CSV (ej: ventas.csv): Ventas reporte
Reporte CSV generado exitosamente en: Ventas reporte.csv
Proceso de exportacion de ventas finalizado.
```

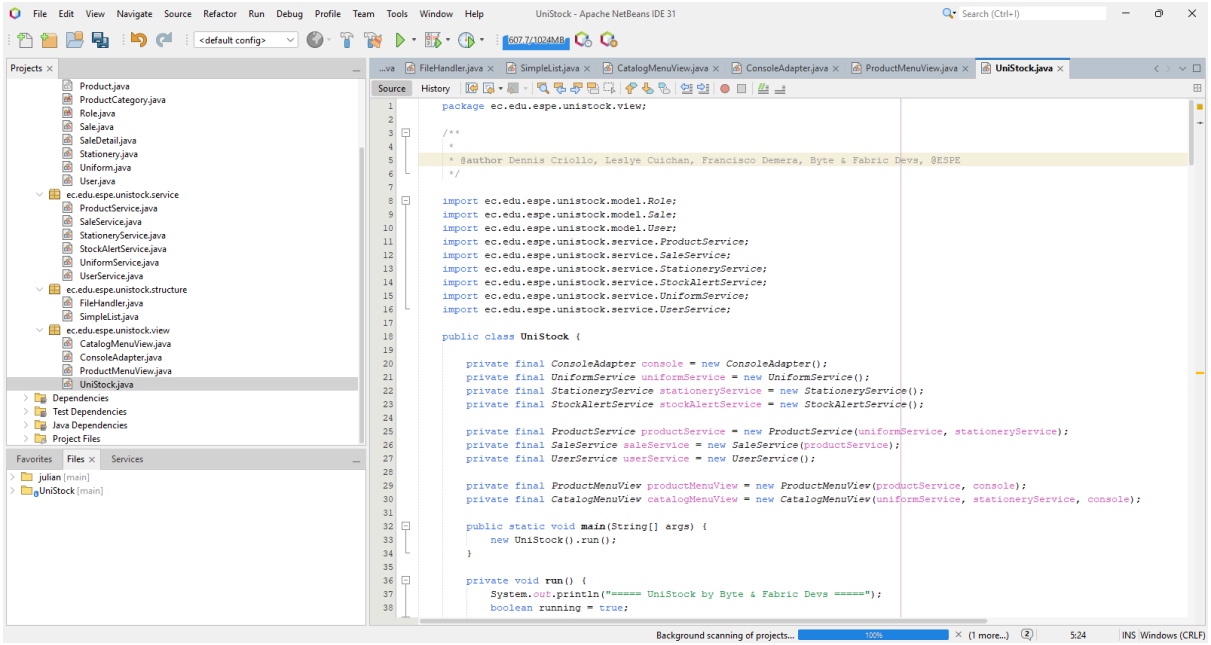
Observations: The system generates the required two CSV data reports (users and sales). It would be a good improvement to include the date in the reports.

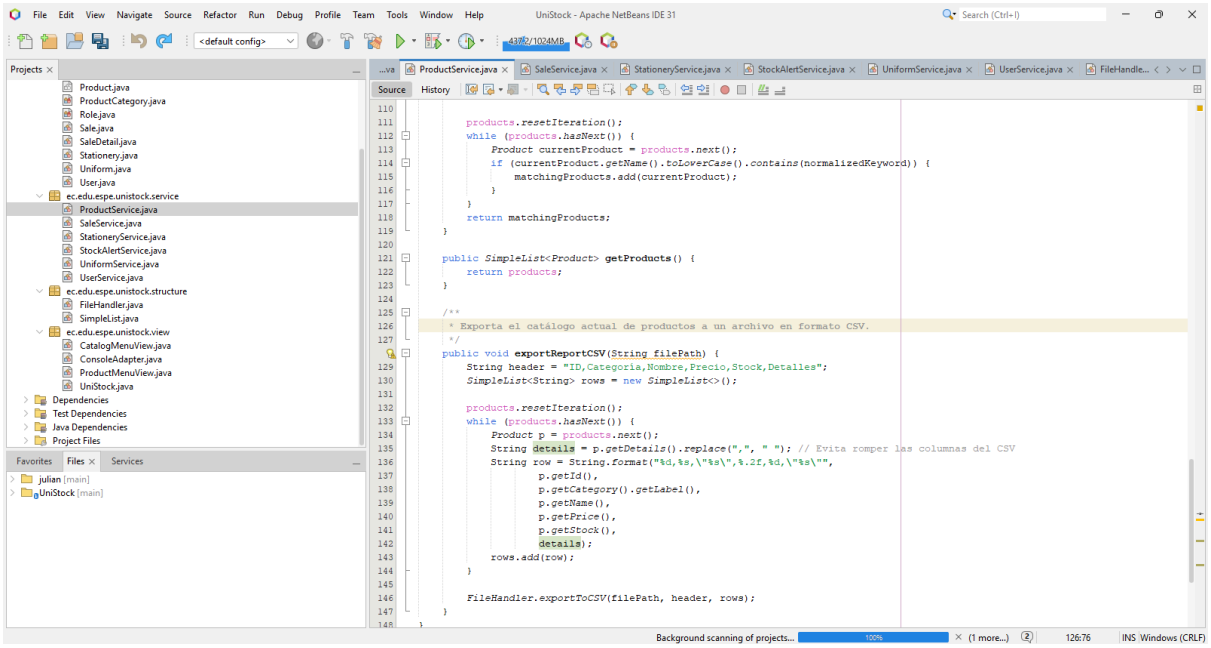
09. Clean Code — 10 points

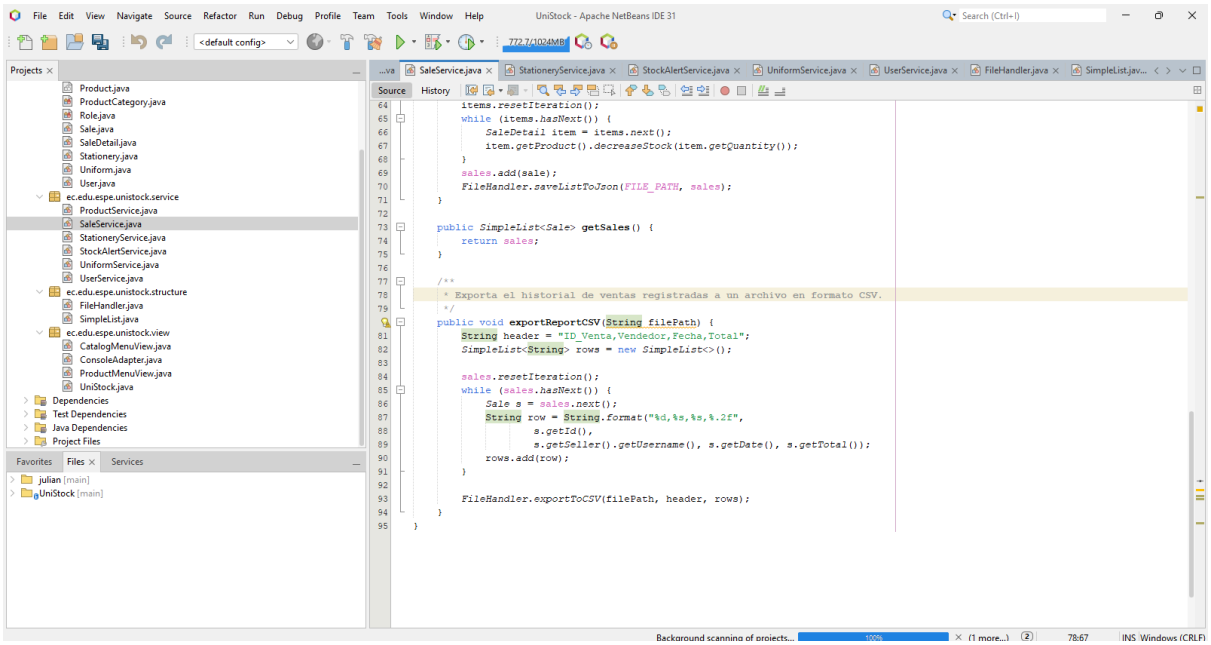
Requirement: List code errors and differences between the code and the UML diagram.

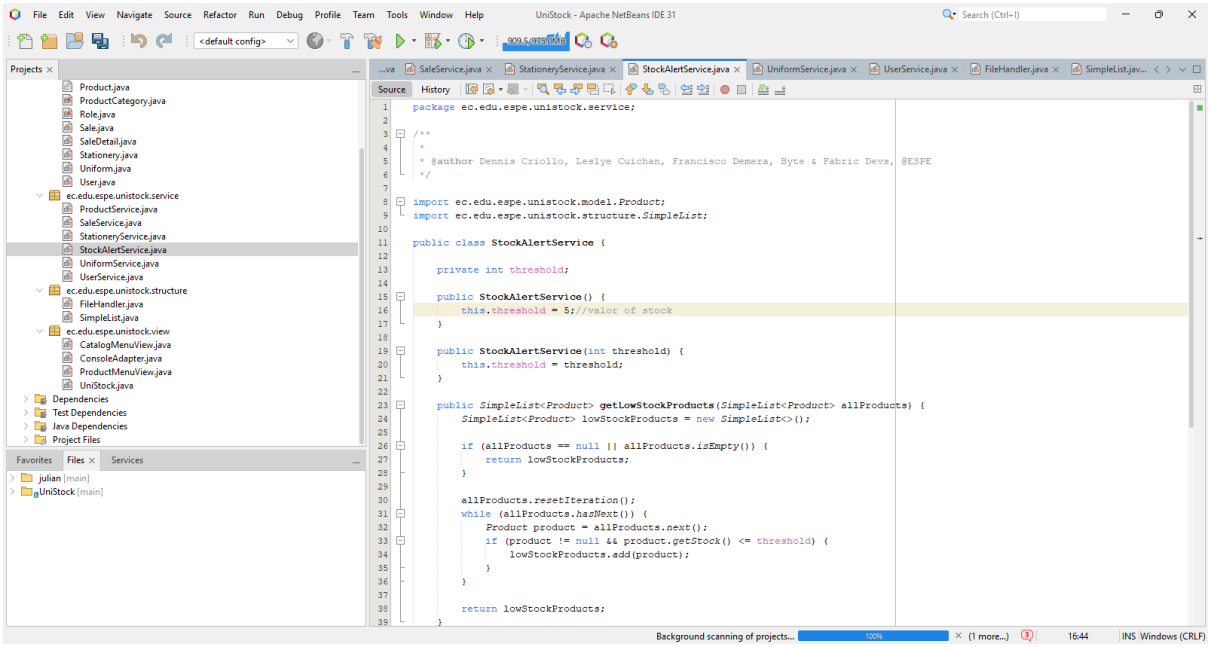
Score: 8.4 / 10

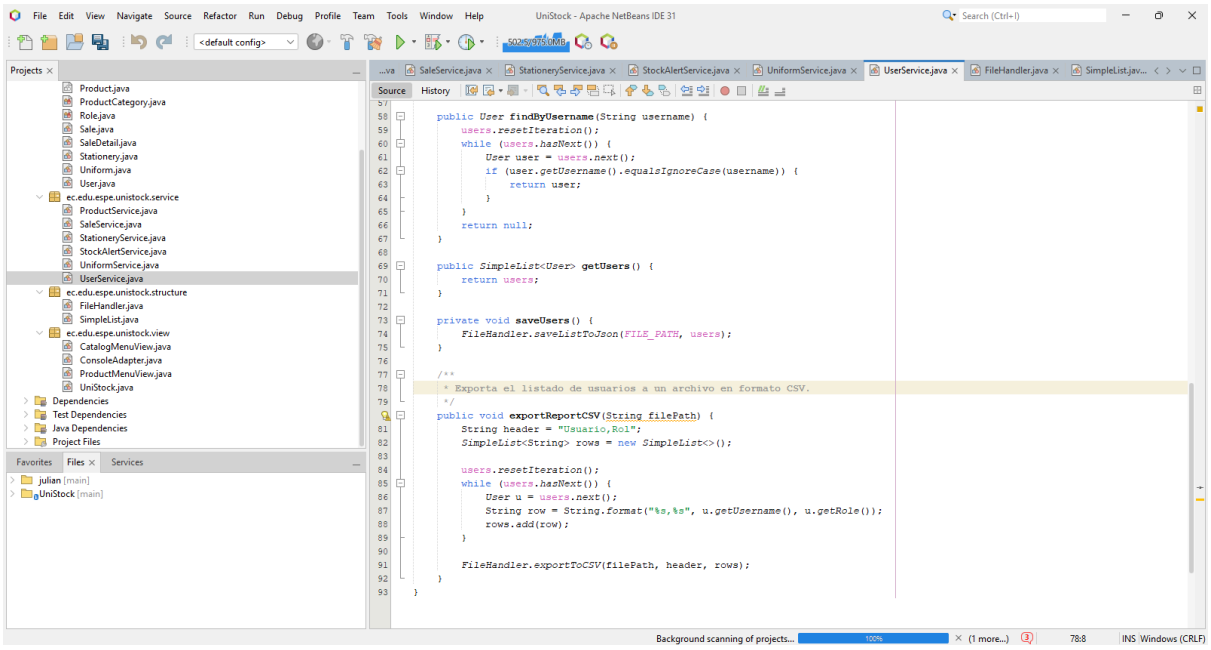
Evidence

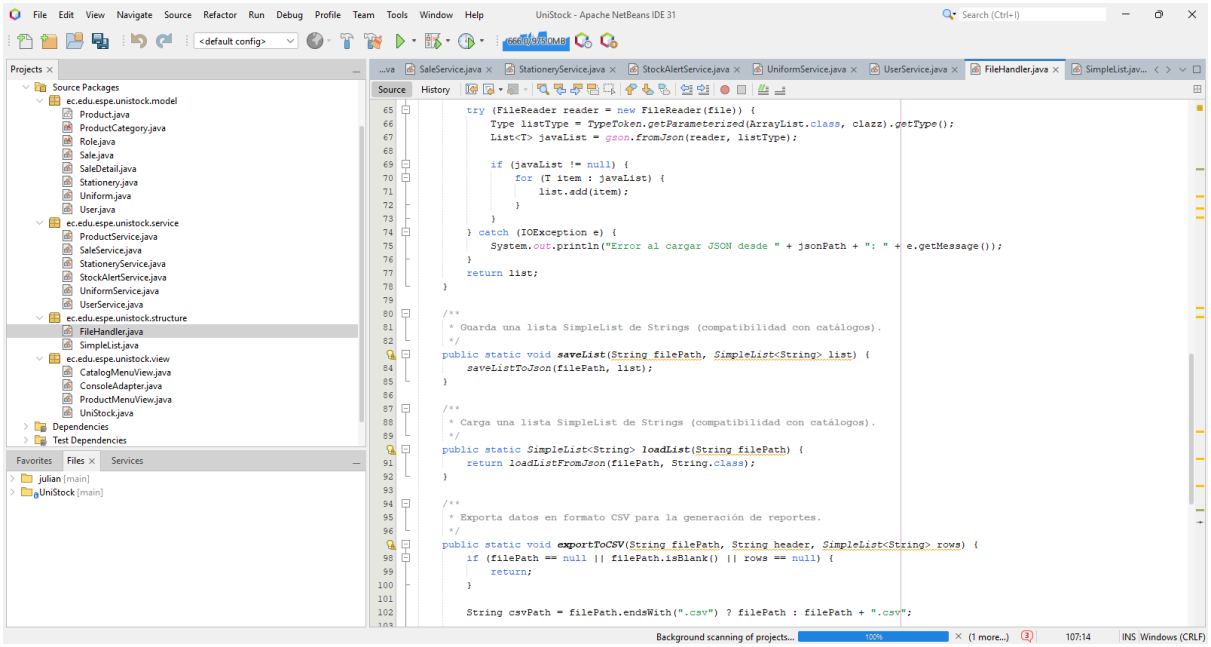



File Edit View Navigate Source Refactor Run Debug Profile Team Tools Window Help
UniStock - Apache NetBeans IDE 31
Search (Ctrl+I)
<default config>
Projects
Product.java
ProductCategory.java
Role.java
Sale.java
SaleDetail.java
Stationery.java
Uniform.java
User.java
ec.edu.espe.unistock.service
ProductService.java
SaleService.java
StationeryService.java
StockAlertService.java
UniformService.java
UserService.java
ec.edu.espe.unistock.structure
FileHandler.java
SimpleList.java
ec.edu.espe.unistock.view
CatalogMenuView.java
ConsoleAdapter.java
ProductMenuView.java
UniStock.java
Dependencies
Test Dependencies
Java Dependencies
Project Files
Favorites
Files
Services
julian [main]
UniStock [main]
ProductService.java
SaleService.java
StationeryService.java
StockAlertService.java
UniformService.java
UserService.java
FileHandle...
Source
History
products.resetIteration();
while (products.hasNext()) {
Product currentProduct = products.next();
if (currentProduct.getName().toLowerCase().contains(normalizedKeyword)) {
matchingProducts.add(currentProduct);
}
}
return matchingProducts;
}
public SimpleList<Product> getProducts() {
return products;
}
/**
* Exporta el catálogo actual de productos a un archivo en formato CSV.
*/
public void exportReportCSV(String filePath) {
String header = "ID,Categoria,Nombre,Precio,Stock,Detalles";
SimpleList<String> rows = new SimpleList<>();
products.resetIteration();
while (products.hasNext()) {
Product p = products.next();
String details = p.getDetails().replace(",", " "); // Evita romper las columnas del CSV
String row = String.format("%d,%s,\"%s\",%.2f,%d,\"%s\"",
p.getId(),
p.getCategory().getLabel(),
p.getName(),
p.getPrice(),
p.getStock(),
details);
rows.add(row);
}
FileHandler.exportToCSV(filePath, header, rows);
}
}
Background scanning of projects...
100%
(1 more...)
126:76
INS Windows (CRLF)


File Edit View Navigate Source Refactor Run Debug Profile Team Tools Window Help
UniStock - Apache NetBeans IDE 31
Search (Ctrl+I)
<default config>
Projects
Product.java
ProductCategory.java
Role.java
Sale.java
SaleDetail.java
Stationery.java
Uniform.java
User.java
ec.edu.espe.unistock.service
ProductService.java
SaleService.java
StationeryService.java
StockAlertService.java
UniformService.java
UserService.java
ec.edu.espe.unistock.structure
FileHandler.java
SimpleList.java
ec.edu.espe.unistock.view
CatalogMenuView.java
ConsoleAdapter.java
ProductMenuView.java
UniStock.java
Dependencies
Test Dependencies
Java Dependencies
Project Files
Favorites
Files
Services
julian [main]
UniStock [main]
SaleService.java
StationeryService.java
StockAlertService.java
UniformService.java
UserService.java
FileHandler.java
SimpleList.jav...
Source
History
items.resetIteration();
while (items.hasNext()) {
SaleDetail item = items.next();
item.getProduct().decreaseStock(item.getQuantity());
}
sales.add(sale);
FileHandler.saveListToJson(FILE_PATH, sales);
}
public SimpleList<Sale> getSales() {
return sales;
}
/**
* Exporta el historial de ventas registradas a un archivo en formato CSV.
*/
public void exportReportCSV(String filePath) {
String header = "ID_Venta,Vendedor,Fecha,Total";
SimpleList<String> rows = new SimpleList<>();
sales.resetIteration();
while (sales.hasNext()) {
Sale s = sales.next();
String row = String.format("%d,%s,%s,%.2f",
s.getId(),
s.getSeller().getUsername(), s.getDate(), s.getTotal());
rows.add(row);
}
FileHandler.exportToCSV(filePath, header, rows);
}
}
Background scanning of projects...
100%
(1 more...)
78:67
INS Windows (CRLF)


File Edit View Navigate Source Refactor Run Debug Profile Team Tools Window Help
UniStock - Apache NetBeans IDE 31
Search (Ctrl+I)
<default config>
Projects
Product.java
ProductCategory.java
Role.java
Sale.java
SaleDetail.java
Stationery.java
Uniform.java
User.java
ec.edu.espe.unistock.service
ProductService.java
SaleService.java
StationeryService.java
StockAlertService.java
UniformService.java
UserService.java
ec.edu.espe.unistock.structure
FileHandler.java
SimpleList.java
ec.edu.espe.unistock.view
CatalogMenuView.java
ConsoleAdapter.java
ProductMenuView.java
UniStock.java
Dependencies
Test Dependencies
Java Dependencies
Project Files
Favorites
Files
Services
julian [main]
UniStock [main]
SaleService.java
StationeryService.java
StockAlertService.java
UniformService.java
UserService.java
FileHandler.java
SimpleList.jav...
Source
History
package ec.edu.espe.unistock.service;
/**
* @author Dennis Criollo, Leslye Cuichan, Francisco Demera, Byte & Fabric Devs, @ESPE
*/
import ec.edu.espe.unistock.model.Product;
import ec.edu.espe.unistock.structure.SimpleList;
public class StockAlertService {
private int threshold;
public StockAlertService() {
this.threshold = 5;//valor of stock
}
public StockAlertService(int threshold) {
this.threshold = threshold;
}
public SimpleList<Product> getLowStockProducts(SimpleList<Product> allProducts) {
SimpleList<Product> lowStockProducts = new SimpleList<>();
if (allProducts == null || allProducts.isEmpty()) {
return lowStockProducts;
}
allProducts.resetIteration();
while (allProducts.hasNext()) {
Product product = allProducts.next();
if (product != null && product.getStock() <= threshold) {
lowStockProducts.add(product);
}
}
return lowStockProducts;
}
Background scanning of projects...
100%
(1 more...)
16:44
INS Windows (CRLF)


File Edit View Navigate Source Refactor Run Debug Profile Team Tools Window Help
UniStock - Apache NetBeans IDE 31
Search (Ctrl+I)
<default config>
Projects
Product.java
ProductCategory.java
Role.java
Sale.java
SaleDetail.java
Stationery.java
Uniform.java
User.java
ec.edu.espe.unistock.service
ProductService.java
SaleService.java
StationeryService.java
StockAlertService.java
UniformService.java
UserService.java
ec.edu.espe.unistock.structure
FileHandler.java
SimpleList.java
ec.edu.espe.unistock.view
CatalogMenuView.java
ConsoleAdapter.java
ProductMenuView.java
UniStock.java
Dependencies
Test Dependencies
Java Dependencies
Project Files
Favorites
Files
Services
julian [main]
UniStock [main]
SaleService.java
StationeryService.java
StockAlertService.java
UniformService.java
UserService.java
FileHandler.java
SimpleList.jav...
Source
History
public User findByUsername(String username) {
users.resetIteration();
while (users.hasNext()) {
User user = users.next();
if (user.getUsername().equalsIgnoreCase(username)) {
return user;
}
}
return null;
}
public SimpleList<User> getUsers() {
return users;
}
private void saveUsers() {
FileHandler.saveListToJson(FILE_PATH, users);
}
/**
* Exporta el listado de usuarios a un archivo en formato CSV.
*/
public void exportReportCSV(String filePath) {
String header = "Usuario,Rol";
SimpleList<String> rows = new SimpleList<>();
users.resetIteration();
while (users.hasNext()) {
User u = users.next();
String row = String.format("%s,%s", u.getUsername(), u.getRole());
rows.add(row);
}
FileHandler.exportToCSV(filePath, header, rows);
}
}
Background scanning of projects...
100%
(1 more...)
78:8
INS Windows (CRLF)



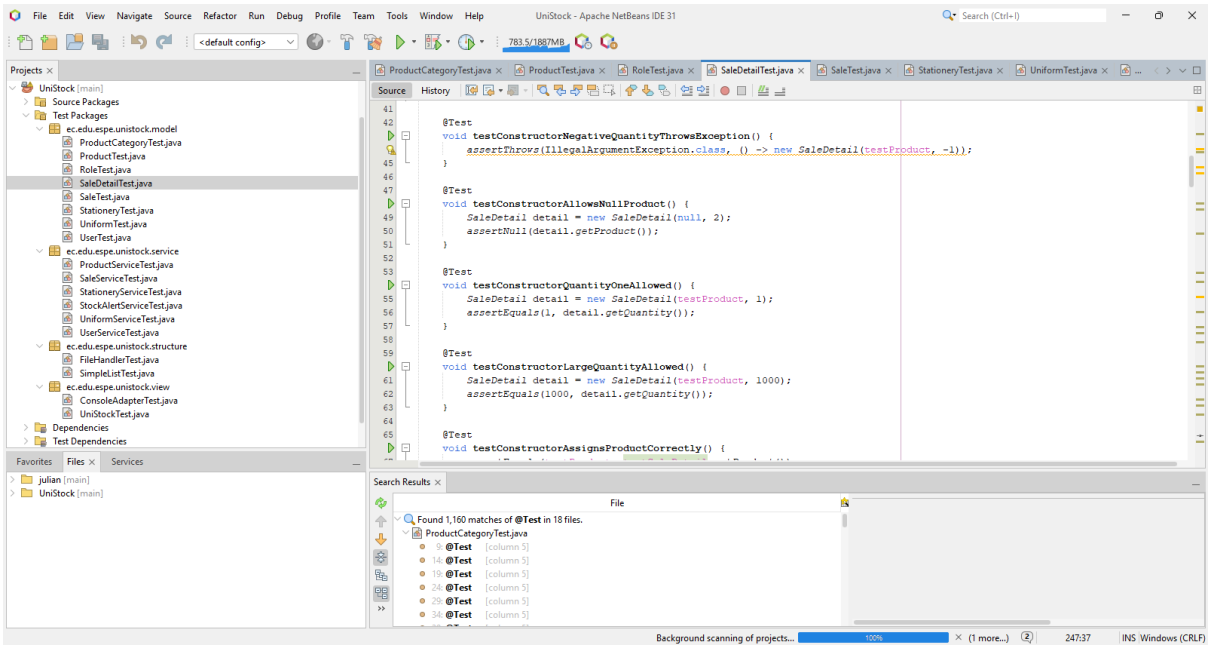
Observations: After reviewing, we found classes containing comments, excluding the default author line generated by the program, and the main menu is disorganized.

10. Test Cases / Unit Tests — 10 points

Requirement: At least 90 unit tests are included.

Score: 10 / 10

Evidence (screenshots, links, or files):



Observations: Upon review, we found over 1,000 tests across 18 classes.

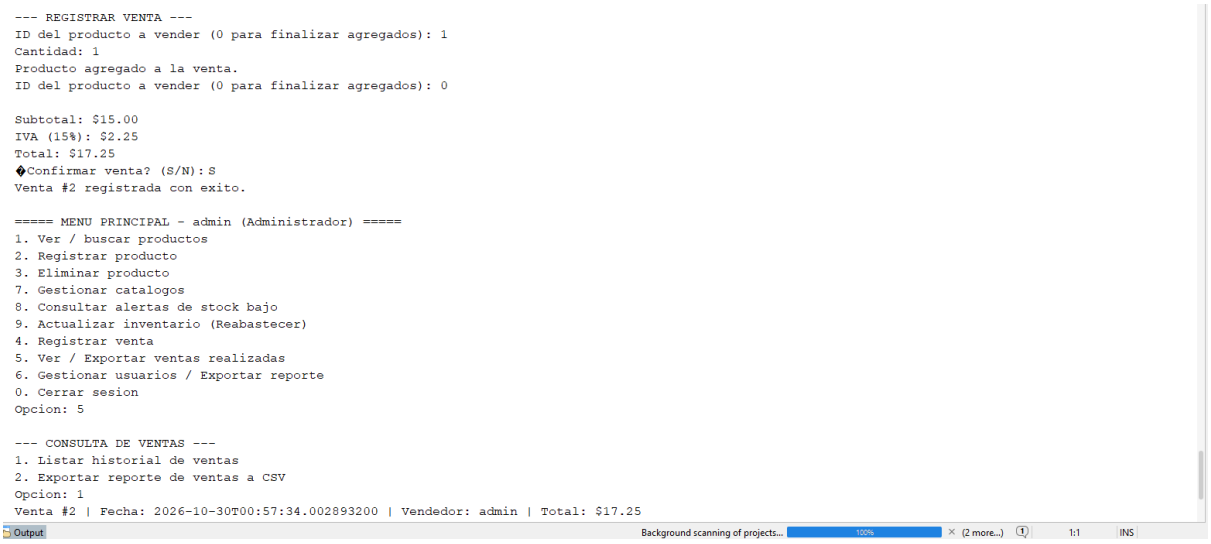
11. JSON Data Files — 10 points

Requirement: The system saves its data and keeps it over time.

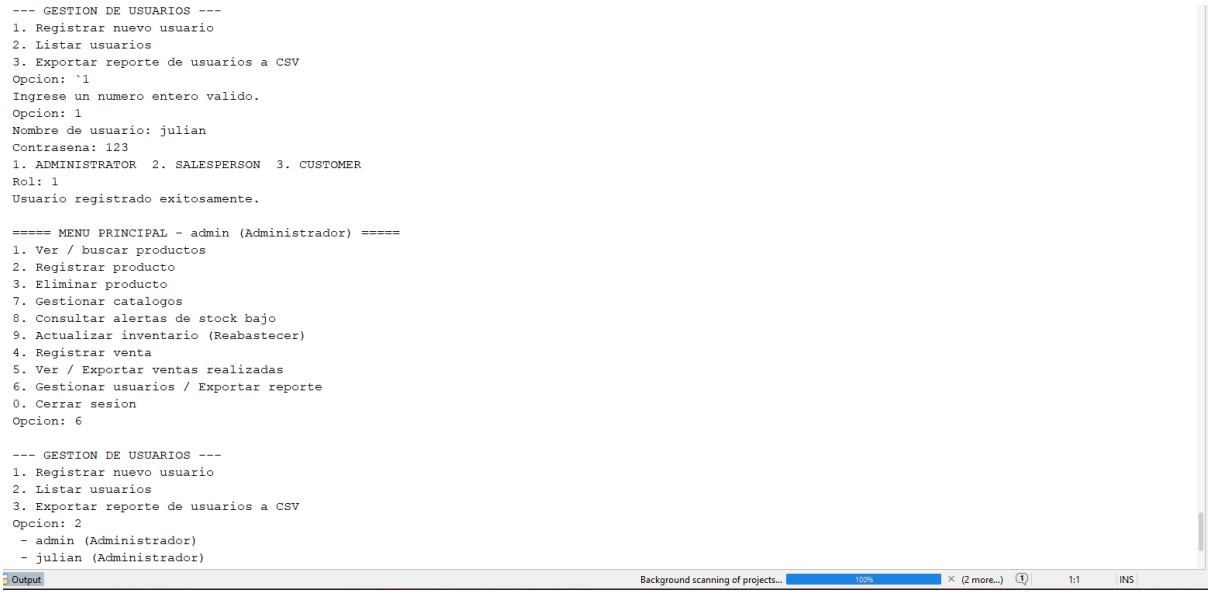
Score: 8.5 / 10

Evidence (screenshots, links, or files):









Observations: We observe that when creating new users, updating inventory, registering products, or recording sales, the data is saved within the program and remains there throughout its entire execution.

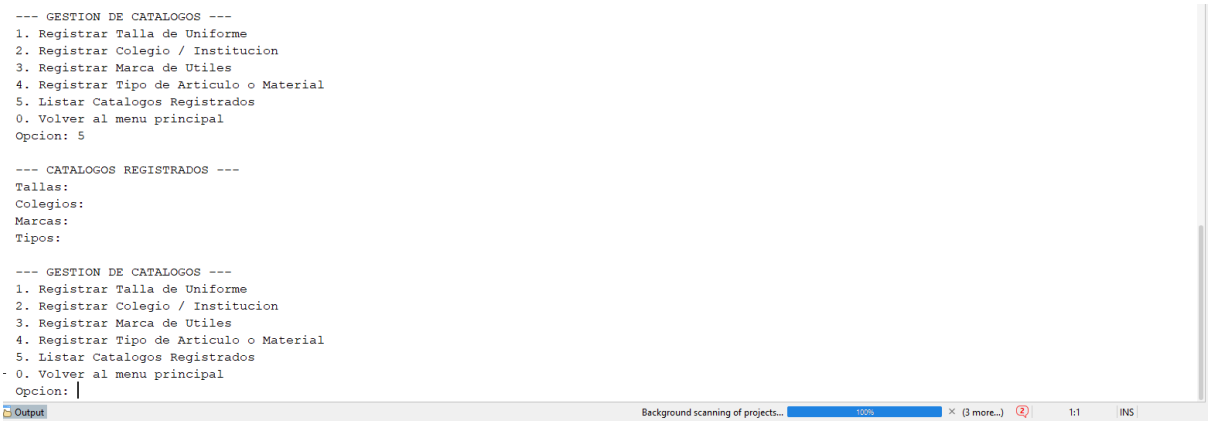
12. Execution of Features — 100 points

Requirement: Run and demonstrate all system features.

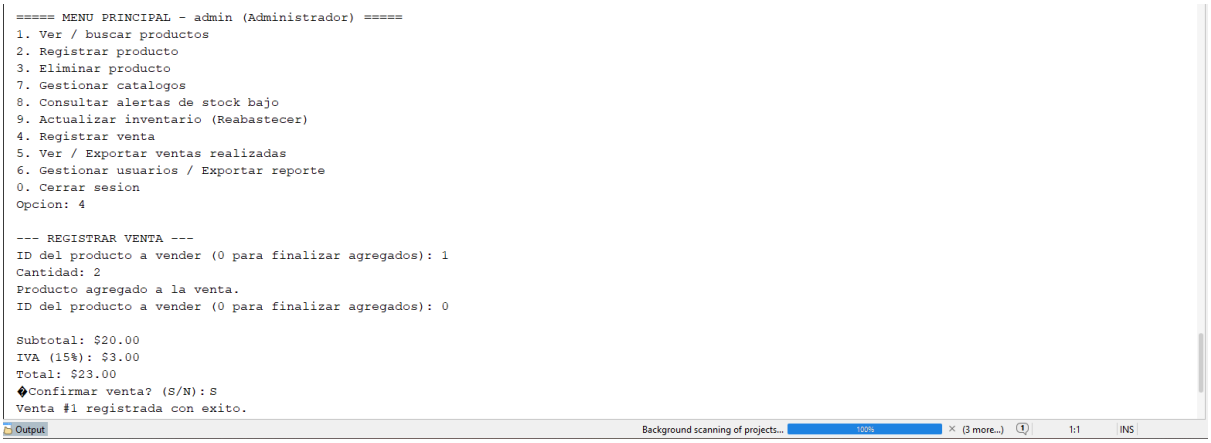
Score: 85 / 100

Evidence (screenshots, links, or files):

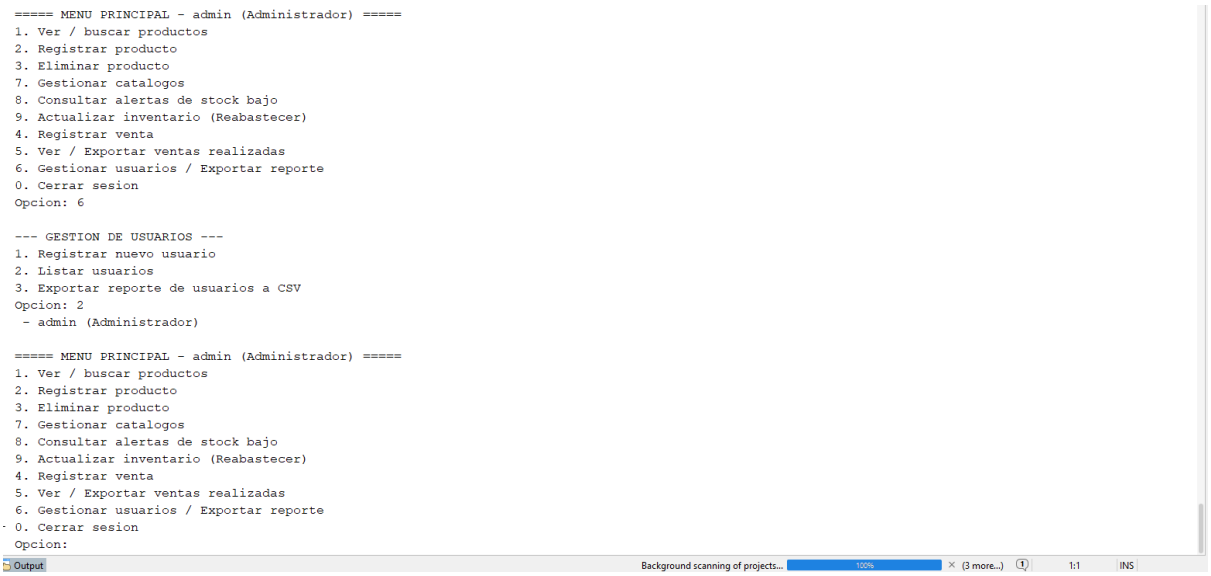
```
===== MENU PRINCIPAL - admin (Administrador) =====
1. Ver / buscar productos
2. Registrar producto
3. Eliminar producto
7. Gestionar catalogos
8. Consultar alertas de stock bajo
9. Actualizar inventario (Reabastecer)
4. Registrar venta
5. Ver / Exportar ventas realizadas
6. Gestionar usuarios / Exportar reporte
0. Cerrar sesion
Opcion: 2
1. Uniforme  2. Papeleria / Util escolar
Categoria: 2
Nombre: lapiz
Precio: 1
Stock inicial: 1000
Marca del producto: Norma
Tipo de articulo / material: Escritura
```




```
===== MENU PRINCIPAL - admin (Administrador) =====
1. Ver / buscar productos
2. Registrar producto
3. Eliminar producto
7. Gestionar catalogos
8. Consultar alertas de stock bajo
9. Actualizar inventario (Reabastecer)
4. Registrar venta
5. Ver / Exportar ventas realizadas
6. Gestionar usuarios / Exportar reporte
0. Cerrar sesion
Opcion: 2
1. Uniforme  2. Papeleria / Util escolar
Categoria: 1
Nombre: Camisa
Precio: 10
Stock inicial: 100
Talla: L
Colegio: Montufar
```







Observations: Upon analyzing the product backlog, we see that the code satisfies the features in the document and the client's requirements.

FINAL EVALUATION

Artifacts	Observations	Grade
100 GitHub		
10 Definition	The document clearly explains the business	10
10 List of possible objects	diagram shows the main business classes	8
10 UML class diagram	The diagram includes classes, attributes, methods, relationships, and multiplicities.	8
10 SRS IEEE 830	The backlog is well organized but can have more specification in some features	9
10 Product Backlog	Defined features	10
10 UML use case diagrams	Some extra features that weren't in the SRS or product backlog	9
10 Data reports CSV	CSV data reports one for users and another for sales but no have a lot of information from beforehand	9
10 Clean code:	Some comments an unorganized menu and in first instance some compilation errors	8.4
10 Test cases/unit test:	More than 1000 tests, excellent	10
10 Data files JSON	The system register every data that the user writes but by fixing code json have been deleted so no many information inside beforehand	8.5

100 Execution of features	The customer requirements are met but some features as menu or data inside can be better	85

TOTAL SCORE OVER 10 : 8.75 / 10